

KHULNA UNIVERSITY OF ENGINEERING & TECHNOLOGY
Department of Computer Science and Engineering

CSE 4102: Computer Graphics and Image Processing Laboratory
4th Year 1st Term

SENTINEL

A Fully Procedural Game Engine and Open-World Military Shooter

Submitted by

Md. Shifat Hasan

Roll: 2107067

Submitted to

Md Tajmilur Rahman

Md. Mubtashim Abrar Nihal

Instructors, Department of Computer Science and Engineering



Department of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna 9203, Bangladesh

October 8, 2026

Abstract

Most video games ship with gigabytes of files made by artists: models, photographs, recorded sounds. *Sentinel* takes the opposite route. It is a complete game engine and an open-world military shooter in which *nothing* is loaded from a model, image or sound file. Every tree, building, soldier, vehicle, surface, cloud and sound is created at start-up from short mathematical recipes and a random seed.

The engine is written in Odin on OpenGL 4.1, the newest version macOS offers (it has no compute shaders), in six layers that may only depend on the layers beneath. A deferred renderer records what each pixel is made of and then lights it with sun, moon and lamps, cascaded shadows, ambient occlusion, reflections, an atmospheric sky with drifting clouds, fog, bloom, tone mapping and anti-aliasing. Surfaces come from 18 recipes built on periodic gradient, warped and cellular noise. The game adds collision, ladders, four vehicles and a helicopter, stealth enemies, synthesised audio and a five-objective mission.

At 1080p the GPU needs 10.9 ms per frame; 281 unit tests, 30 GL checks, 8 318 texture checks (exact tiling of every material), 318 567 renderer checks and two scripted playthroughs pass. Each technique is explained in plain language with its governing equations.

Contents

	PAGE
Abstract	i
Contents	ii
List of Tables and Figures	iii
CHAPTER I Introduction	1
1.1 Introduction	1
1.2 Background and Problem Statement	1
1.3 Objectives	1
1.4 Scope	2
1.5 Unfamiliarity of the Problem and its Solution	2
1.6 Project Planning and Work Distribution	2
1.7 Applications of the Work	2
1.8 Organization of the Project	2
CHAPTER II Related Works	3
2.1 Introduction	3
2.2 Related Works	3
2.3 Discussion	3
CHAPTER III Methodology	4
3.1 Introduction	4
3.2 Detailed Methodology	4
3.3 Conclusion	6
CHAPTER IV Implementation, Results and Discussions	7
4.1 Introduction	7
4.2 Experimental Setup	7
4.3 Evaluation Metrics	7
4.4 Dataset	7
4.5 Implementation and Results	7
4.6 Objective Achieved	9
4.7 Conclusion	9
CHAPTER V Societal, Health, Environment, Safety, Ethical, Legal and Cultural Issues	10
5.1 Intellectual Property Considerations	10
5.2 Ethical Considerations	10
5.3 Safety Considerations	10
5.4 Legal Considerations	10
5.5 Impact on Societal, Health and Cultural Issues	10
5.6 Impact on the Environment and Sustainability	11
CHAPTER VI Addressing Complex Engineering Problems and Activities	12
6.1 Complex Engineering Problems	12
6.2 Complex Engineering Activities	12
CHAPTER VII Conclusions	13
7.1 Summary	13
7.2 Limitations	13
7.3 Recommendations and Future Works	13
References	13

List of Tables

Table No.	Description	Page
2.1	Comparison with related approaches (typical)	3

List of Figures

Figure No.	Description	Page
1.1	Work plan by day.	2
3.1	Six layers and the main loop.	4
3.2	Gradient, fractal, warped, ridged and cellular noise (illustration of the shader equations).	5
3.3	The passes of one frame.	6
4.1	GPU time per pass (total 10.91 ms of 16.7 ms).	8
4.2	The engine and game in action; (h) and (i) show the streak bug and its fix. .	8

CHAPTER I

Introduction

1.1 Introduction

Imagine a film studio that buys no props: a recipe book tells assistants how to build every tree, wall and cloud, and a different “seed” number gives a different but believable forest. *Sentinel* is that studio for a game: a *game engine* (the machinery that draws, simulates and sounds a world) plus a first-person military mission. *Procedural* means shapes, colours and sounds are calculated, not stored; the **GPU** (a processor with thousands of tiny cores) runs small **shader** programs that paint each pixel.

1.2 Background and Problem Statement

Graphics courses teach lighting, textures and noise separately; products need them working *together* at 60 frames per second, usually with art loaded from files. The problem: *can a complete, good-looking, playable 3D game be produced by code alone on a student laptop?* Two limits: no model, image or precomputed file is allowed, and macOS stops at OpenGL 4.1, without compute shaders.

1.3 Objectives

1. Build a layered engine with downward-only dependencies.
2. Generate all meshes, textures and sounds from code and seeds.
3. Implement a modern renderer (PBR shading, shadows, sky, post-processing).
4. Build a playable game with vehicles, ladders, doors and a mission.
5. Verify it with tests and a 16.7 ms frame budget.

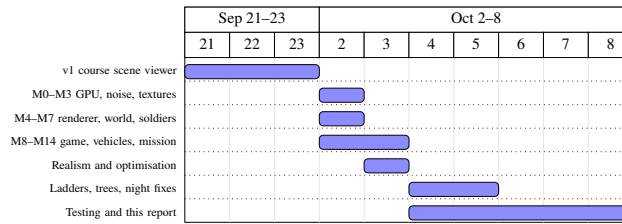


Figure 1.1: Work plan by day.

1.4 Scope

Tools: Odin, OpenGL 4.1 core with GLSL shaders (47 files), GLFW (window, input), miniaudio (sound output), Git (138 commits) and $\text{\LaTeX}$ /TikZ. Networking and hand-made art are out of scope.

1.5 Unfamiliarity of the Problem and its Solution

The design follows published theory, not an existing code base. Engines usually import art; *.kkrieger* shows procedural shooters are possible but is not readable teaching code. Here “no files” is enforced on every feature and each file holds one feature.

1.6 Project Planning and Work Distribution

One student did all the work (design, code, tests, report). Figure 1.1 is rebuilt from the 138 commits.

1.7 Applications of the Work

A teaching tool for procedural graphics and a base for small source-only games.

1.8 Organization of the Project

Chapters II–IV: related work, design, results; V–VI: social and engineering issues; VII: conclusions.

Table 2.1: Comparison with related approaches (typical)

Feature	Tutorials	Commercial	.kkrieger	Sentinel
No imported art	Mixed	No	Yes	Yes
Complete game	No	Yes	Yes	Yes
Day/night, clouds, shadows	Rare	Yes	No	Yes
Written to be read	Yes	No	No	Yes

CHAPTER II

Related Works

2.1 Introduction

Sentinel combines noise, physically based shading, real-time pipelines and procedural games.

2.2 Related Works

Perlin noise [1], Worley cellular noise [2] and domain warping [13] give natural patterns. Cook–Torrance with GGX [3–5] and Oren–Nayar [8] model reflection. Deferred shading [14], cascaded shadows [7], Hillaire’s sky [6], ACES [9], dithering [10] and FXAA [11] complete the pipeline. *.kkrieger* (2004) generated a whole shooter at run time; *Project I.G.I.* (2000) inspired the mission.

2.3 Discussion

Table 2.1: tutorials teach one technique, commercial engines assume imported art, demo shooters are not readable.

CHAPTER III

Methodology

3.1 Introduction

One frame is an assembly line: the game decides what exists, the engine builds shapes and surfaces, the renderer paints them, 60 times a second.

3.2 Detailed Methodology

3.2.1 Overall Framework

Figure 3.1: a lower layer never knows about a higher one. Main.odin is a table of contents (initialise, loop, shut down); procedures stay near 40 lines and every GL call is checked.

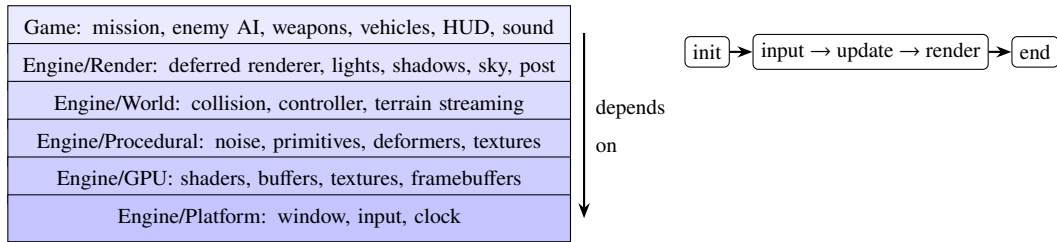


Figure 3.1: Six layers and the main loop.

3.2.2 Procedural Geometry and Characters

An object is a *table* of parts: primitive, transform, deformers, material. A noise deformer moves each vertex along its normal $\hat{n}$ by at most the amplitude A , so a sphere becomes a rock:

$$\mathbf{p}' = \mathbf{p} + A \nu(f\mathbf{p}) \hat{n}, \quad \|\mathbf{p}' - \mathbf{p}\| \leq A. \quad (3.1)$$

Soldiers are capsule skeletons; legs reach targets by two-bone IK, $\cos \theta = (a^2 + b^2 - c^2)/2ab$.

3.2.3 Procedural Textures

A *recipe* is a small shader answering “what colour, bumpiness and shininess is here?”, baked once at start-up into a 1024^2 picture (22 materials).

In plain words

Gradient noise is a rubber sheet pinned at grid points with random slopes: smooth, never exactly repeating. *Fractal noise* adds finer sheets on top, like hills, boulders and pebbles.

Fractal noise sums octaves, each at double frequency and half strength; because grid gradients are chosen from the cell index modulo the period P , the picture tiles without a seam, $F(\mathbf{x} + P\mathbf{e}) = F(\mathbf{x})$ (checked exactly by tests):

$$F(\mathbf{x}) = \frac{\sum_{i=0}^{n-1} 2^{-i} \nu(2^i \mathbf{x})}{\sum_{i=0}^{n-1} 2^{-i}}. \quad (3.2)$$

Warping $F(\mathbf{x} + s[F_1, F_2])$ gives stone, ridged noise $(1 - |\nu|)^2$ cracks and bark, cellular noise leaves (Fig. 3.2).

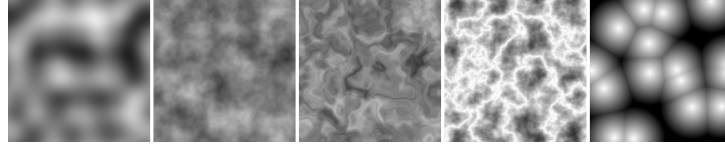


Figure 3.2: Gradient, fractal, warped, ridged and cellular noise (illustration of the shader equations).

A height map becomes a *normal map* that fakes bumps. Slopes use the Scharr filter, the most direction-fair 3×3 kernel, and crevices (below their neighbours’ average) are darkened:

$$h_x \approx \frac{3(h_{NE} + h_{SE} - h_{NW} - h_{SW}) + 10(h_E - h_W)}{32 \Delta}, \quad \hat{n} \propto (-kh_x, -kh_y, 1). \quad (3.3)$$

Triplanar mapping (project along three axes, blend by $|n_i|^k$) stops deformed shapes stretching textures.

3.2.4 The Rendering Pipeline

In plain words

Deferred shading photographs a “data poster” of the scene first (each pixel’s colour, direction and shininess), then lights the poster once, which makes many lights cheap.

Figure 3.3 shows the passes. Direct light uses Cook–Torrance [3]: D is how many tiny mirror facets face the viewer (GGX), G how much they shade each other, F the glancing-angle reflection (Schlick), α the roughness squared:

$$f_s = \frac{D F G}{4(\mathbf{n} \cdot \mathbf{l})(\mathbf{n} \cdot \mathbf{v})}, \quad D = \frac{\alpha^2}{\pi((\mathbf{n} \cdot \mathbf{h})^2(\alpha^2 - 1) + 1)^2}, \quad F = F_0 + (1 - F_0)(1 - \mathbf{v} \cdot \mathbf{h})^5. \quad (3.4)$$

Distant surfaces fade into haze, $C = TC_{\text{surf}} + (1 - T)C_{\text{haze}}$ with $T = e^{-kd}$, and the HDR image is compressed by the ACES curve [9], graded, dithered and smoothed (FXAA).



Figure 3.3: The passes of one frame.

3.2.5 Sky, Terrain, Physics and Gameplay

The sun’s height follows astronomy, $\sin(\text{elev}) = \sin \varphi \sin \delta + \cos \varphi \cos \delta \cos h$; the sky colour comes from Rayleigh and Mie scattering [6], with a hashed star field at night. Clouds sit on planes at height H (1500 m cumulus, 7 km cirrus); a view ray with direction $\mathbf{d}$ meets the plane at $\mathbf{P}_{xz} = \frac{H}{d_y}(d_x, d_z) + \mathbf{w}t$, so shifting the noise by wind $\mathbf{w}t$ makes clouds drift.

Terrain is warped, ridged fractal noise flattened into a plateau for the base and streamed in 64 m chunks. **Trees** (oak, pine, birch, cypress) are tables of limbs and leaf lumps; each instance gets its own size, lean and tint and bends in the wind as a cantilever, $\Delta \propto (y/9)^2$. **Collision** uses rotatable boxes and a capsule player. Cars follow the bicycle model $\dot{\psi} = v \tan \delta / L$ with power fading toward top speed, $\dot{v} = a_0(1 - 0.6|v|/v_{\text{max}})$. **Ladders** are a four-state machine (none, mounting, climbing, topping out; 1.1 m/s up, 1.5 m/s down) in which the controller owns the body, so it cannot get stuck. A staged objective machine drives five mission goals; all sounds are synthesised.

3.3 Conclusion

Small data-driven techniques give a deterministic world from seeds.

CHAPTER IV

Implementation, Results and Discussions

4.1 Introduction

This chapter states how the work was measured and shows what it produces.

4.2 Experimental Setup

An Apple-silicon Mac (macOS, OpenGL 4.1 core), release build (-o:speed): 164 Odin files (about 19 200 lines) and 47 GLSL files (about 2 000 lines). Benchmark: a scripted 1 km flight, 1080p, 300 frames.

4.3 Evaluation Metrics

Frame time $t = 1000/\text{fps}$ ms (budget 16.7 ms), GPU time per pass, test pass/fail, and tiling error $\max |f(\mathbf{u} + \mathbf{k}) - f(\mathbf{u})|$, which must be zero.

4.4 Dataset

None. All content comes from fixed seeds, so every run is reproducible.

4.5 Implementation and Results

4.5.1 Quantitative results

The display locks the average frame to 16.75 ms (60 Hz); the GPU work needs 10.91 ms (Fig. 4.1), leaving headroom. Verification: 281 unit tests (noise, meshes, IK, collision,

ladders, vehicles, mission), 30 GL checks, 8 318 texture checks (exact tiling of every material), 318 567 renderer checks (lighting, shadows, atmosphere), and two scripted playthroughs that both finish 5 of 5 objectives.

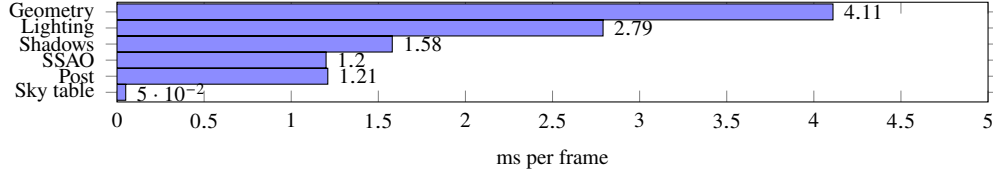


Figure 4.1: GPU time per pass (total 10.91 ms of 16.7 ms).

4.5.2 Qualitative results

Figure 4.2 shows the running game: the base, a firefight with alarm, the automatic material showroom, IK-posed soldiers, tree species, drifting clouds and hands on a ladder. Panels (h) and (i) document a bug: the fog colour sampled the whole sky, so a horizon star painted a vertical line on every surface in its column (left of centre); the fog now uses the smooth atmosphere only.

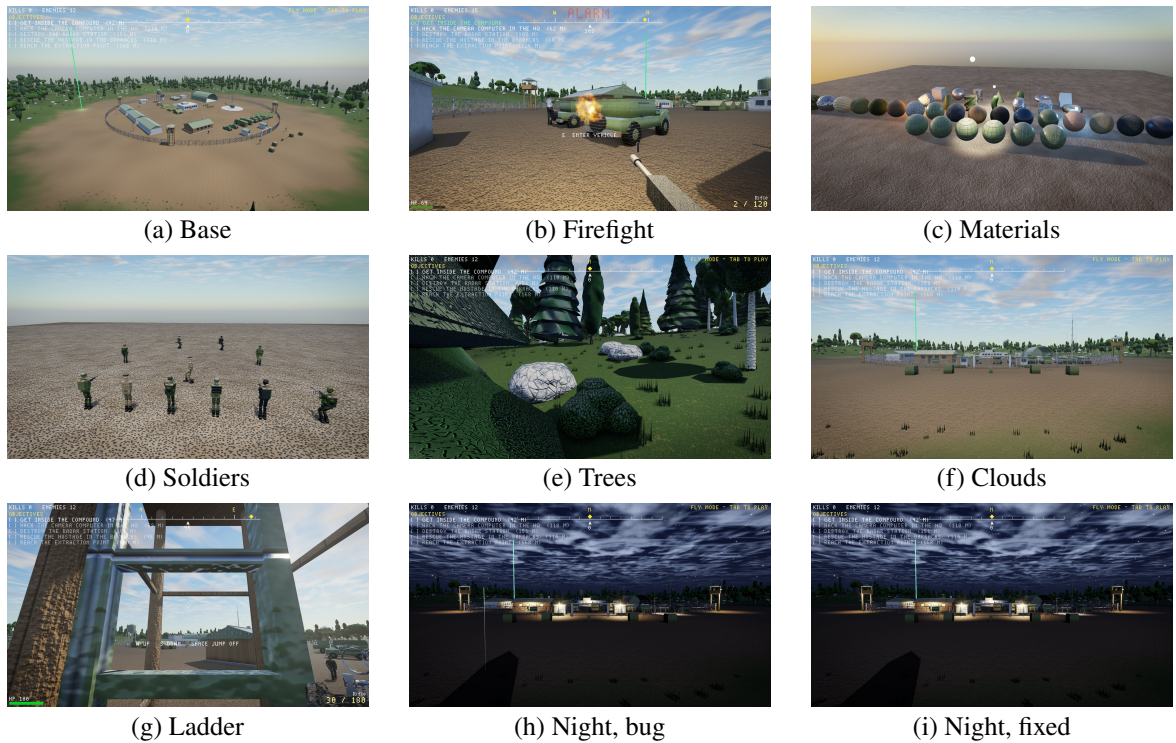


Figure 4.2: The engine and game in action; (h) and (i) show the streak bug and its fix.

4.5.3 Analysis of the results

Geometry costs most, so instancing and per-chunk culling matter. The streak was found by bisecting: removing draw items, lights and effects until it vanished.

4.6 Objective Achieved

(1) Six layers with downward-only dependencies. (2) No model, image or sound file: 22 materials, 4 tree species, 5 vehicles, synthesised audio. (3) Deferred PBR with shadows, SSAO, reflections, sky, clouds, bloom, ACES, FXAA. (4) Two playable missions with ladders, vehicles and stealth. (5) 281 tests, 327 000 GL checks, 2 playthroughs, 10.9 ms GPU time.

4.7 Conclusion

A procedural engine meets a 60 fps budget and passes exact checks.

CHAPTER V

Societal, Health, Environment, Safety, Ethical, Legal and Cultural Issues

5.1 Intellectual Property Considerations

All content is original; libraries are open source. The mission is inspired by, not copied from, *Project I.G.I.*

5.2 Ethical Considerations

Fiction for adults; no personal data.

5.3 Safety Considerations

Short flashes, limited sound.

5.4 Legal Considerations

Licences respected; violence stylised; sources cited.

5.5 Impact on Societal, Health and Cultural Issues

Lowers the barrier to learning graphics; rescue and stealth goals temper shooting.

5.6 Impact on the Environment and Sustainability

No art downloads; display sync saves GPU work.

CHAPTER VI

Addressing Complex Engineering Problems and Activities

6.1 Complex Engineering Problems

Depth of knowledge: radiometry, noise, filters, GPU pipelines, state machines. **Conflicts:** realism versus a 16.7 ms frame; no files versus rich surfaces; no compute shaders. **Analysis:** bisecting the star streak, proving tiling. **Interdependence:** renderer, world, game and audio share data and timing.

6.2 Complex Engineering Activities

The project used modern tools, integrated many components and verified quality by measurement.

CHAPTER VII

Conclusions

7.1 Summary

Sentinel shows that a complete 3D game, from surfaces and shapes to weather and sound, can be generated entirely by code under an OpenGL 4.1 limit: a six-layer engine, a deferred physically based renderer, noise-based recipes, table-driven objects and a mission game, in about 21 000 lines within a 60 fps budget.

7.2 Limitations

Models are blocky; there is no true global illumination; foliage is lumpy volumes, not leaves; the beacon is a solid column; tuning used scripts and screenshots, not a large playtest.

7.3 Recommendations and Future Works

Leaf cards with cut-out transparency, screen-space global illumination, point-light shadows, multiplayer and a user study; a modern graphics API with compute shaders.

REFERENCES

- [1] K. Perlin, “Improving noise,” *ACM Trans. Graph. (SIGGRAPH)*, vol. 21, no. 3, pp. 681–682, 2002.
- [2] S. Worley, “A cellular texture basis function,” in *Proc. SIGGRAPH*, 1996, pp. 291–294.
- [3] R. L. Cook and K. E. Torrance, “A reflectance model for computer graphics,” *ACM Trans. Graph.*, vol. 1, no. 1, pp. 7–24, 1982.
- [4] B. Walter, S. R. Marschner, H. Li and K. E. Torrance, “Microfacet models for refraction through rough surfaces,” in *Proc. Eurographics Symp. Rendering*, 2007, pp. 195–206.
- [5] B. Karis, “Real shading in Unreal Engine 4,” SIGGRAPH Course: Physically Based Shading in Theory and Practice, 2013.
- [6] S. Hillaire, “A scalable and production ready sky and atmosphere rendering technique,” *Computer Graphics Forum*, vol. 39, no. 4, pp. 13–22, 2020.
- [7] R. Dimitrov, “Cascaded shadow maps,” NVIDIA White Paper, 2007.
- [8] M. Oren and S. K. Nayar, “Generalization of Lambert’s reflectance model,” in *Proc. SIGGRAPH*, 1994, pp. 239–246.
- [9] K. Narkowicz, “ACES filmic tone mapping curve,” blog post, 2016.

- [10] J. Jimenez, "Next generation post processing in Call of Duty: Advanced Warfare," SIGGRAPH Course: Advances in Real-Time Rendering, 2014.
- [11] T. Lottes, "FXAA," NVIDIA White Paper, 2009.
- [12] M. Segal and K. Akeley, *The OpenGL Graphics System: A Specification, Version 4.1*, Khronos Group, 2010.
- [13] I. Quilez, "Domain warping," online article, iquilezles.org.
- [14] S. Hargreaves and M. Harris, "Deferred shading," NVIDIA presentation, Game Developers Conference, 2004.
- [15] H. Scharf, "Optimal operators in digital image processing," Ph.D. dissertation, Univ. Heidelberg, 2000.
- [16] T. Akenine-Möller, E. Haines and N. Hoffman, *Real-Time Rendering*, 4th ed., CRC Press, 2018.